Dashayla Fields

CS 320

10/19/2026

Title: Summary and Reflection Report:

(Software Testing for Mobile Application Services)

Summary

For this project, I created JUnit tests for the Contact, Task, and Appointment services. My goal was to make sure each class followed the requirements before moving on to the next part of the project. For the Contact service, I tested that contact IDs were unique, names stayed within the required character limits, and fields could not be null. I also tested the update methods to verify that contact information could be changed correctly after a contact was created. For the Task service, I used a similar approach. I checked that task IDs remained unique, and that task names and descriptions met the length requirements. I also tested the service methods used to add, update, and delete tasks and lastly for Appointment service, I focused on validating appointment IDs, descriptions, and dates. One of the most important requirements was preventing appointments from being scheduled in the past, so I created tests to verify that invalid dates would cause exceptions.

My testing approach closely matched the project requirements because each rule listed in the assignment had a corresponding test. For example, contacts could not have IDs longer than ten characters, so I created tests that attempted to create contacts with IDs over the limit and verified that exceptions were thrown. The same strategy was used for tasks and appointments. Any requirement involving field length, null values, uniqueness, or dates was tested directly. This helped me confirm that the code was performing exactly as the project specifications required.

I believe my JUnit tests were effective because they covered both valid and invalid scenarios. Instead of only checking that the program worked when correct values were entered, I also tested situations where users might enter incorrect information. The tests helped identify issues during

development before they became bigger problems. By covering constructors, validation logic, update methods, and service operations, I had confidence that most of the application's functionality was being exercised during testing. Although code coverage percentages are useful, I found it more valuable to focus on whether every requirement had been tested. A high coverage percentage means little if the important requirements are not being validated. Writing the JUnit tests was one of the more challenging parts of the project because it forced me to think differently about the code. Instead of focusing on making the application work, I had to focus on finding ways it could fail. As I worked through each class, I became more comfortable creating test cases and anticipating potential issues. The process helped me better understand the importance of testing and how it contributes to software quality. I ensured the code was technically sound by creating tests that verified both expected behavior and exception handling. For example, I used assertions to confirm that invalid data would not be accepted and that valid data was stored correctly. These Examples included using assertions such as `assertEquals()` to verify returned values and `assertThrows()` to confirm that exceptions occurred when invalid data was passed into constructors and update methods. The tests allowed me to verify that validation logic worked as intended and that objects behaved correctly after updates were performed. I tried to keep my tests efficient by having each test focus on one specific requirement. When one test failed, it was easier to identify the source of the problem because each test only evaluated a single behavior. I also avoided repeating unnecessary code by reusing object creation where appropriate and organizing my tests so they were easy to read and maintain. This made debugging faster and improved the overall quality of the test suite.

The main testing technique I used was unit testing. Unit testing allowed me to test individual classes and methods without worrying about how they interacted with larger systems. This made it easier to identify problems early in development. I also used boundary value testing. Many

requirements involved character limits, so I tested values at the maximum allowed length as well as values that exceeded the limit. This helped ensure that validation rules worked properly. Another technique I used was exception testing. Instead of only testing successful situations, I intentionally used invalid values to verify that the application rejected them when necessary. One testing technique I did not use was integration testing. Since this project focused mainly on individual service classes, I was not testing interactions with databases, web services, or user interfaces. I also did not perform system testing. System testing evaluates an entire application from the user's perspective and would be more useful in a larger project that includes a complete front end and back end. Performance testing was another technique that was not necessary for this project. The application was relatively small and did not have the kind of workload that would require measuring speed or scalability. Unit testing is useful during development because it helps identify problems quickly and keeps bugs isolated within specific components. Boundary testing is important whenever an application has limits on user input because errors often happen at those boundaries. Integration testing becomes more important in larger projects where different components must communicate with each other. System testing is valuable before deployment because it validates the complete application. Performance testing is useful when software must support many users or process large amounts of data.

While working on this project, I tried to think like someone looking for problems instead of someone hoping the code would work. That mindset helped me create stronger tests because I was constantly looking for edge cases and invalid inputs. Being cautious was important because even a small mistake in validation logic could affect multiple parts of the application. For example, if contact IDs were not validated correctly, the entire Contact Service could end up

storing invalid records. The same was true for appointment dates and task information. One way I tried to limit bias was by testing situations where I expected the code to fail instead of only testing situations where it should succeed. This forced me to view the project from the perspective of a user who might enter bad data. If I were only testing my own code without considering failures, it would be easy to overlook mistakes because I already know how the application is supposed to work. Testing invalid inputs helped me avoid making assumptions and made my tests more objective. This project showed me that writing code is only part of software development. Testing is equally important because it ensures that the application works reliably and continues to work as changes are made. Cutting corners during testing might save time initially, but it often creates more work later when defects are discovered. Technical debt can build quickly when issues are ignored or when testing is incomplete. As a future software engineer, I plan to reduce technical debt by writing tests alongside development, reviewing requirements carefully, and making sure important functionality is validated before considering a feature complete. Investing time in quality early is much easier than fixing major issues after software has already been released.

REFRENCES

JUnit Team. (n.d.). *JUnit 5 user guide*. JUnit 5. https://junit.org/junit5/

Oracle. (n.d.). *Java documentation*. Oracle. https://docs.oracle.com/